

design principle, client code should only be concerned with the interface of a class and should not care about the implementation of the methods. Consider the example of a simple `Person` class in [Listing 8-1](#).

```
class Person():  
  
    def __init__(self, name, salary):  
        self.name = name  
        self.salary = salary
```

Listing 8-1: Data ownership in the `Person` class

The values of the instance variables

`self.name` and `self.salary` are set whenever we instantiate new `Person` objects, like this:

```
oPerson1 = Person('Joe Schmoe', 90000)  
oPerson2 = Person('Jane Smith', 99000)
```

Each `Person` object owns its own set of the two instance variables.

Interpretations of Encapsulation

Here is where things get a little controversial. Different programmers have different views about the accessibility of an instance variable. Python provides for a loose interpretation of encapsulation by allowing direct access to instance variables using simple dot syntax. Client code can legally access an instance variable of an object by name using the syntax `<object>.<instanceVariableName>`.

However, a *strict* interpretation of encapsulation says that client software should never be able to retrieve or change the value of an instance variable directly. Instead, the only way that a client can retrieve or change a value held in an object is to use a method provided by the class for this purpose.

Let's look at both approaches.